

SwasthyaFlow

Product Requirements Document (PRD)

Status: Phase 1 MVP & Phase 2 Enhanced Prototype

Project Type: AI SaaS | Multimodal Healthcare Triage & Referral Support Platform

Target Users: Government Hospitals, PHCs, Health Camps, Institutional Clinics, Nurses, Health Workers, Medical Officers

Tagline: AI-assisted frontline triage. Human-led clinical decisions.

1. Vision

Frontline healthcare facilities often receive patient information through conversations, regional-language speech, text, and medical reports. Organizing this information manually can consume valuable healthcare-worker time, especially in high-load facilities.

SwasthyaFlow is a multimodal AI-assisted triage workflow that converts patient-provided information into a structured, reviewer-facing triage note. It helps qualified healthcare staff organize symptoms, timelines, report information, missing information, follow-up questions, review-priority signals, and referral information.

The product is explicitly non-diagnostic. It must not prescribe treatment, replace a qualified professional, or present AI output as a clinical decision.

2. Problem Statement

Build a human-in-the-loop healthcare triage assistant that helps government hospitals, primary health centers, public health camps, company clinics, industrial-estate health units, and campus health centers summarize patient-provided symptoms, uploaded reports, and basic visual inputs into a structured triage note for qualified review.

The solution should be designed for India-wide contexts where patient load, language diversity, specialist availability, and digital maturity vary significantly across facilities.

3. Product Goals

- Reduce manual effort required to organize patient-provided information.
- Convert voice, text, and medical reports into structured reviewer-facing information.
- Support English, Hindi, and Odia in the demonstration.
- Extract key details and construct symptom timelines.
- Identify missing or unclear information instead of inventing it.
- Generate concise follow-up questions for healthcare workers.
- Provide configurable review-priority signals through a rules-based safety layer.
- Provide a human reviewer dashboard and escalation workflow.
- Generate structured referral notes for higher-level review.
- Demonstrate consent, anonymization, minimal data retention, auditability, and role-based access.
- Keep all health-related AI output advisory and reviewer-facing.

4. Target Audience

Primary Users

- Nurses
- Health Workers
- Medical Officers
- Doctors / Qualified Clinical Reviewers

Facility Types

- Government Hospitals
- Primary Health Centres
- Public Health Camps
- Company Clinics
- Industrial-estate Health Units
- Campus Health Centres

5. Success Metrics

Metric	Target for Prototype
Voice-to-structured intake	Complete a demo case within approximately 30 seconds after input.
Report processing	Extract relevant synthetic report fields with visible source information.
Triage generation	Generate structured triage card within approximately 15 seconds where model/API latency permits.
Missing information	Identify intentionally omitted fields in test cases.
Safety workflow	Configured urgency scenarios are consistently surfaced by the rules engine.
Human review	Reviewer can edit every AI-generated field before completion.
Referral	Generate a structured referral note in one action.
Privacy	Consent, anonymization, role access and audit trail visible in demo.

6. Evaluation Criteria Alignment

Criterion	Weight
Safety-first triage workflow	20%
Quality of information extraction and summarization	20%
Multimodal capability: text, voice, OCR, image understanding	15%
India-wide facility relevance and accessibility	15%
Human-review design and escalation logic	15%
Privacy and responsible AI controls	10%
Demo Quality	5%

Development priorities should follow the supplied evaluation weightage.

7. Product Scope

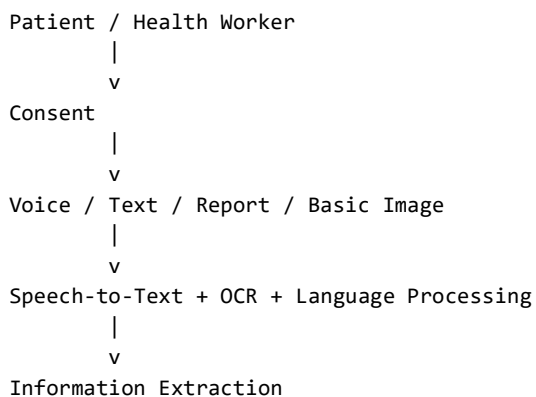
Phase 1 — MVP

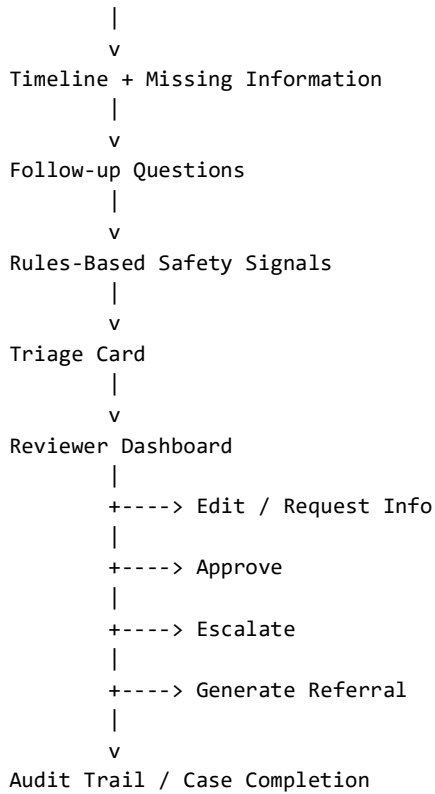
- Patient intake
- Consent
- Anonymous patient ID
- Text input
- Voice input
- Speech-to-text
- English/Hindi/Odia demonstration
- Medical report upload
- OCR
- Structured information extraction
- Timeline generation
- Missing information detection
- Follow-up questions
- Rules-based review priority
- Triage card
- Reviewer queue
- Human edit/approve/escalate
- Referral note
- Audit trail
- Synthetic demo data

Phase 2 — Enhanced Prototype

- Smart image-quality check
- OCR confidence / uncertain-text highlighting
- Source-aware extraction
- Offline / low-connectivity simulation
- Facility configuration
- Reviewer analytics
- Export / print
- Additional Indian-language architecture
- More configurable safety rules
- Enhanced audit and reviewer comparison

8. Core User Journey





9. Patient Intake

The health worker creates an encounter using minimum necessary information.

Field	Required	Notes
Anonymous Patient ID	Yes	Generated automatically, e.g. P-1042.
Age	Yes	Numeric.
Sex	Optional	Only if required by workflow.
Preferred language	Yes	English / Hindi / Odia for MVP.
Chief complaint	Yes	Short patient-reported reason for visit.
Symptoms	Yes	Patient-reported information only.
Duration / onset	Optional	Mark unknown if unavailable.
Basic observations	Optional	Only values explicitly entered by staff.
Medication information	Optional	Only if supplied.
Consent	Yes	Must be captured before processing.

10. Consent & Privacy

Before AI processing, display a concise consent screen explaining the educational prototype purpose, information processing, AI limitations, and human review requirement.

- Use anonymous patient IDs.
- Do not use real patient records.
- Collect only information needed for the triage workflow.

- Keep source information distinguishable from AI-generated summaries.
- Show an audit trail of major processing and reviewer actions.
- Use role-based access in the prototype.
- Do not expose health-related output as a diagnosis or treatment recommendation.

11. Multimodal Input

Text Input

Allow the health worker to enter patient-reported information manually.

Voice Input

Allow a patient or health worker to record speech. Demonstration languages: Hindi and Odia, with English as a supported working language.

Voice -> Speech-to-Text -> Language Detection -> Translation/Normalization -> Structured Extraction

Medical Report / Image Input

Allow upload or camera capture of synthetic medical reports and basic visual inputs.

Image/Document -> Quality Check -> OCR/Vision Extraction -> Structured Data -> Human Verification

12. Speech-to-Text

- Record audio from browser.
- Show recording state.
- Allow stop, replay and retry.
- Display transcript before final processing.
- Allow healthcare worker to edit obvious transcription errors.
- Preserve original transcript.
- Do not silently invent words when audio is unclear.

13. Translation & Language Layer

The MVP should demonstrate English, Hindi and Odia. Preserve original patient language and show translated/normalized text separately.

Layer	Example
Original	Patient's Hindi/Odia speech or transcript
Normalized	Clean text for information extraction
Structured	Reviewer-facing English summary
Source	Original input remains accessible

14. Medical Report OCR

The report pipeline should convert a synthetic uploaded report into structured information while preserving the original image/document.

1. Upload image/document.
2. Run image-quality checks.

3. Preprocess when required.
4. Run OCR or image-text extraction.
5. Preserve raw OCR.
6. Extract relevant fields.
7. Highlight uncertain extraction.
8. Allow reviewer correction.
9. Store the final reviewed structure.

Never claim that an extracted value is a diagnosis. Use wording such as: "Value may require clinical review."

15. Image Processing

- Auto resize
- Auto rotate
- Grayscale conversion
- Contrast enhancement
- Noise reduction
- Adaptive thresholding
- Text sharpening when required
- Image-quality warning for blurry, poorly lit, cropped or tilted images

Purpose: improve OCR/readability and prevent avoidable extraction errors.

16. Information Extraction

Extract only information explicitly present in patient input or uploaded documents.

- Symptoms
- Onset
- Duration
- Patient-reported severity when stated
- Frequency when stated
- Relevant history explicitly mentioned
- Current medications explicitly mentioned
- Relevant report values
- Timeline
- Missing information
- Ambiguous information
- Potential configured safety signals

17. Timeline Engine

Convert free-form descriptions into a concise chronological timeline.

Example:

Day -2: Fever reported as beginning.

Day -1: Weakness reported as increasing.

Today: Dizziness reported.

Source: Patient voice

18. Missing Information Engine

A key differentiator is that the system must identify what it does not know.

State	Meaning
KNOWN	Explicitly supplied by patient, staff or document.
UNKNOWN	Not supplied.
UNCLEAR	Mentioned but insufficiently specific.

- Current temperature
- Current BP
- Current pulse
- Medication history
- Exact symptom duration
- Other workflow-specific missing fields

19. Follow-up Question Generator

Generate short, relevant questions for a healthcare worker. Questions should target missing or unclear information and must not be diagnostic.

Example:
Reported concern: chest discomfort

- Suggested questions:
1. When did it begin?
 2. Where is it located?
 3. Is it continuous or intermittent?
 4. Has it become worse?
 5. Is there difficulty breathing?
 6. Has fainting occurred?

20. Safety / Review-Priority Engine

The LLM must not directly determine a clinical diagnosis or treatment. Use a configurable rules-based layer to identify predefined urgency signals.

Priority	UI Meaning
GREEN	Routine review — no configured urgency signal detected.
YELLOW	Priority review — configured signal requires relatively prompt professional review.
RED	Urgent review — potential urgent signal detected and requires urgent qualified-staff review.

Required disclaimer: “AI-generated review priority. Not a medical diagnosis. Final assessment must be made by qualified healthcare staff.”

21. Rules Engine

Example Rule
{

```

"rule_id": "R001",
"signal": "reported_fainting",
"priority": "RED",
"message": "Reported fainting requires urgent professional review."
}

```

Rules must flag signals, not diagnose diseases.

- Rules should be deterministic and configurable.
- Rules should consume structured facts rather than raw model guesses wherever possible.
- Every triggered rule should have an explanation.
- Reviewer can inspect the source information behind the signal.
- Final clinical responsibility remains with qualified staff.

22. Triage Card — Primary Product Screen

Section	Content
Patient	Anonymous ID, age, language, encounter status
Chief Concerns	Patient-reported symptoms
Timeline	Chronological events
Report Data	Extracted values with source
Missing Information	Unknown / unclear fields
Follow-up Questions	Questions for health worker
Review Priority	GREEN / YELLOW / RED
Safety	Non-diagnostic disclaimer
Actions	Edit, Request Information, Approve, Escalate, Referral

23. Reviewer Dashboard

The reviewer dashboard should look like a practical healthcare queue rather than a generic analytics dashboard.

Patient	Language	Input	Priority	Status	Waiting
P-1021	Odia	Voice + Report	RED	Waiting	08 min
P-1022	Hindi	Voice	YELLOW	Waiting	05 min
P-1023	Odia	Text	GREEN	Reviewed	—

- Filter by priority
- Filter by status
- Filter by language
- Sort by priority and waiting time
- Open triage card
- View source information

24. Human Review

- Reviewer can edit AI-generated fields.
- Reviewer can accept the structured summary.
- Reviewer can request additional information.
- Reviewer can escalate.
- Reviewer can generate a referral note.
- Nothing is considered clinically final merely because AI generated it.

- Record reviewer actions in the audit trail.

25. Source Transparency

Each important field should expose its source where practical.

Fever -> Patient voice
Hb 9.2 g/dL -> Uploaded CBC report
Age 42 -> Health worker entry
Review flag -> Rules Engine R001

26. Referral Note Generator

Generate a structured administrative/referral summary for qualified staff.

REFERRAL SUMMARY

Patient ID:
Age:
Preferred Language:

Reason for Referral:

Reported Symptoms:

Timeline:

Relevant Available Information:

Uploaded Reports:

Missing Information:

Follow-up / Review Questions:

AI Review Priority:

Attachments:

Reviewer Status:

AI-assisted administrative summary. Clinical assessment and referral decision remain with qualified healthcare staff.

The referral generator must never prescribe treatment or state a definitive diagnosis.

27. Reviewer Actions & Statutes

Action	Result
Approve	Marks AI-organized information as reviewed by the authorized reviewer.
Edit	Stores reviewer corrections.
Request Information	Returns the case to the health-worker workflow.
Escalate	Marks case for higher-level professional review.
Generate Referral	Creates a structured referral document.
Complete	Closes the current workflow after required review.

28. Audit Trail

09:42 Encounter created
09:43 Consent recorded
09:43 Voice transcription completed
09:44 OCR completed
09:44 AI extraction completed
09:44 Safety rules evaluated
09:45 Reviewer opened case
09:47 Reviewer edited summary
09:48 Referral generated

Audit records should identify the actor, action, encounter, timestamp and relevant metadata.

29. Role-Based Access

Role	Permissions
Health Worker	Create patient encounter, capture input, upload reports, view/edit intake.
Nurse	Review triage information, ask/request information, escalate.
Medical Officer	Review, edit, approve, escalate, generate referral.
Administrator	Manage users/facility configuration and audit access.

30. Database Design

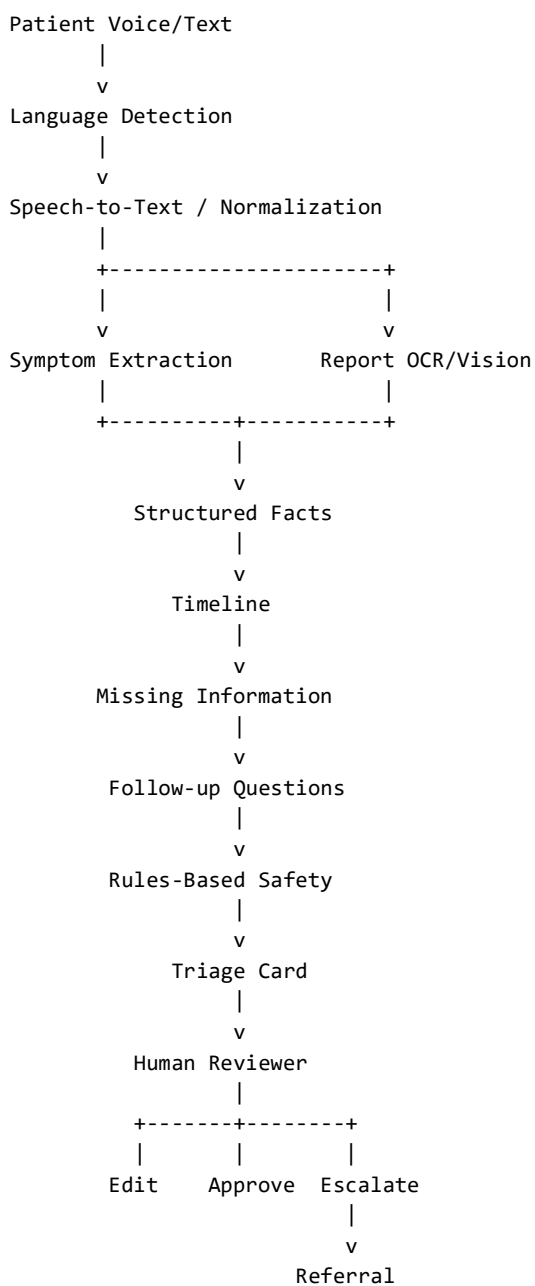
Table	Fields
users	id UUID, name, role, facility_id, created_at
facilities	id UUID, name, type, location, created_at
patients	id UUID, anonymous_patient_id, age, sex, preferred_language, consent_status, created_at
encounters	id UUID, patient_id, chief_complaint, symptoms, timeline, priority, status, created_at
inputs	id UUID, encounter_id, type, original_text, transcript, language, created_at
reports	id UUID, encounter_id, file_url, raw_ocr, extracted_data, quality_status, created_at
triage_notes	id UUID, encounter_id, summary, missing_information, follow_up_questions, risk_signals, priority, created_at
reviews	id UUID, encounter_id, reviewer_id, changes, decision, notes, created_at
referrals	id UUID, encounter_id, content, created_at
audit_logs	id UUID, user_id, encounter_id, action, timestamp, metadata

31. API Design

- POST /api/patients
- POST /api/encounters
- POST /api/speech/transcribe

- POST /api/translate
- POST /api/reports/upload
- POST /api/reports/ocr
- POST /api/triage/analyze
- POST /api/triage/questions
- POST /api/triage/priority
- GET /api/queue
- GET /api/encounters/:id
- PATCH /api/encounters/:id
- POST /api/reviews
- POST /api/referrals
- GET /api/audit/:encounterId

32. AI Pipeline



33. Master AI Prompt

You are an information organization assistant for a healthcare triage workflow.

You are NOT a doctor and must not diagnose, prescribe, recommend treatment, or replace a qualified healthcare professional.

Analyze only information explicitly provided by the patient, healthcare worker, or uploaded document.

Extract:

1. Patient-reported symptoms
2. Symptom onset
3. Symptom duration
4. Timeline
5. Patient-reported severity when explicitly stated
6. Relevant history explicitly mentioned
7. Current medications explicitly mentioned
8. Relevant uploaded-report values
9. Missing information
10. Ambiguous information
11. Reviewer-facing follow-up questions

Rules:

- Never invent information.
- Never infer a diagnosis.
- Never prescribe treatment.
- Never claim the patient has a disease.
- Distinguish patient-reported, document-extracted and AI-generated information.
- Mark unavailable information as unknown.
- Preserve uncertainty.
- Keep output concise and reviewer-facing.
- Do not determine final review priority; the configured rules engine does that.
- Return valid structured JSON only.

34. Structured AI Output

```
{
  "patient_reported": {
    "symptoms": [],
    "duration": [],
    "history": [],
    "medications": []
  },
  "timeline": [],
  "report_data": [],
  "missing_information": [],
  "ambiguous_information": [],
  "follow_up_questions": [],
  "risk_signals": [],
  "review_priority": "PENDING_RULE_ENGINE"
}
```

35. Frontend Screens

10. Landing / Facility Selection
11. Consent & Privacy
12. New Patient Intake

13. Voice Capture
14. Transcript Review
15. Report Upload
16. Image Quality Check
17. OCR / Extraction Review
18. AI Processing Status
19. Triage Card
20. Reviewer Queue
21. Case Review
22. Referral Generator
23. Audit & Privacy
24. Settings / Facility Configuration

36. Page-by-Page Requirements

Landing

- SwasthyaFlow branding
- Start New Patient
- Reviewer Dashboard
- Facility identifier
- Prototype disclaimer

Consent

- Purpose
- AI limitations
- Human review statement
- Consent checkbox
- Continue button

Patient Intake

- Anonymous ID
- Age
- Sex if needed
- Language
- Chief complaint
- Voice/Text entry
- Basic observations
- Continue

Voice Capture

- Large microphone control
- Recording timer
- Transcript
- Replay
- Retry
- Edit transcript
- Confirm

Report Upload

- Drag/drop
- Camera/upload
- Supported file types
- Preview
- Quality status
- Analyze

Triage Card

- Summary
- Timeline
- Report values
- Missing information
- Questions
- Priority
- Sources
- Reviewer actions

Reviewer Queue

- Priority filters
- Waiting time
- Status
- Language
- Open case

Case Review

- Edit AI output
- Compare source
- Review safety signals
- Approve
- Escalate
- Request information
- Generate referral

Audit

- Consent event
- AI processing events
- Reviewer changes
- Referral event
- Timestamps

37. UI Principles

- Premium but practical healthcare product.
- Clean, minimal and spacious.
- Desktop-first reviewer experience.
- Responsive tablet/desktop layout.
- Mobile-friendly intake.
- High contrast and accessible typography.

- Clear distinction between AI output and human decisions.
- Use badges for status and priority.
- Use skeleton loaders during AI/OCR processing.
- Use graceful error states.
- Avoid excessive animations and decorative medical imagery.

38. Component Structure

```
components/  
  ui/  
  layout/  
  intake/  
  voice/  
  reports/  
  ocr/  
  triage/  
  reviewer/  
  queue/  
  referral/  
  privacy/  
  audit/  
  common/  
  
lib/  
  ai/  
  ocr/  
  speech/  
  translation/  
  safety/  
  database/  
  auth/  
  audit/  
  
app/  
  page.tsx  
  intake/  
  voice/  
  reports/  
  triage/  
  reviewer/  
  referral/  
  audit/  
  settings/
```

39. Recommended Technology Stack

Layer	Recommended Technology
Frontend	Next.js + TypeScript
UI	Tailwind CSS + shadcn/ui + Lucide React
Forms	React Hook Form + Zod
Backend	Next.js Route Handlers / Server Actions or FastAPI
Database	PostgreSQL
ORM	Drizzle ORM or equivalent
OCR	Tesseract OCR / PaddleOCR
AI	Lightweight LLM with structured JSON output
Speech	Speech-to-text model/API
Storage	Secure object storage for synthetic report files

Auth	Role-based authentication
Deployment	GitHub-connected web deployment

40. Environment Variables

DATABASE_URL=
 AI_API_KEY=
 SPEECH_TO_TEXT_API_KEY=
 TRANSLATION_API_KEY=
 STORAGE_BUCKET=
 AUTH_SECRET=

Only include variables for services actually used. Never hard-code secrets into frontend code.

41. Non-Functional Requirements

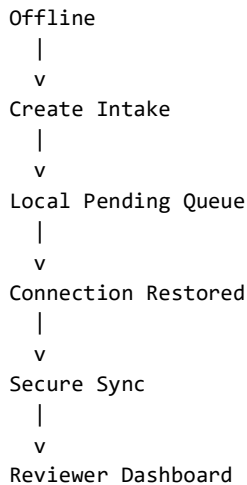
- Responsive on desktop and tablets.
- Mobile-friendly intake experience.
- End-to-end type safety.
- Modular architecture.
- Reusable UI components.
- Graceful OCR, AI, speech and database error handling.
- AI processing should show progress and recover from transient failures.
- Dashboard should feel responsive and avoid unnecessary blocking operations.
- Every AI-generated field must remain editable.
- Source information should remain accessible.
- No real patient records.
- Auditability for major workflow events.

42. Error & Edge States

Scenario	Required Behavior
Blurry report	Warn user and request better image.
OCR uncertain	Highlight uncertain text and require review.
Voice unclear	Allow replay/re-record and mark uncertain transcript.
Unsupported language	Preserve original input and clearly show unsupported processing state.
AI unavailable	Allow manual intake/review workflow to continue.
Rules engine failure	Do not invent a priority; show priority as unavailable/pending.
Missing information	Display unknown/unclear instead of guessing.
Upload failure	Show retry option.
Database failure	Show safe error state; do not claim record was saved.

43. Offline / Low-Connectivity Mode

The target environment includes facilities with varying digital maturity. The prototype should simulate or implement a basic offline workflow.



Clearly label records as Pending Sync until synchronization is confirmed.

44. Synthetic Demo Dataset

Use only synthetic/public sample data.

Patient ID	Age	Language	Input	Priority
P-1001	28	Odia	Voice	GREEN
P-1002	46	Hindi	Voice + CBC	YELLOW
P-1003	62	Odia	Voice + Report	RED
P-1004	34	English	Text + Report	GREEN
P-1005	51	Hindi	Voice	YELLOW

45. Demo Scenario

Use one coherent fictional patient journey for the main presentation.

25. Patient gives a Hindi/Odia voice description.
26. System transcribes and shows the original transcript.
27. System produces a normalized/translated working version.
28. Healthcare worker uploads a synthetic medical report.
29. OCR extracts report values and displays source information.
30. AI generates symptom summary and timeline.
31. System identifies missing information.
32. System generates follow-up questions.
33. Rules engine produces a review priority based on configured signals.
34. Reviewer opens the triage card.
35. Reviewer edits one AI-generated field to demonstrate human control.
36. Reviewer approves or escalates the case.
37. System generates a referral note.
38. Audit screen shows the full processing/review trail.

46. 5-Minute Judge Demo Script

Time	Demo
0:00–0:30	Explain frontline problem: patient load, language

	diversity, reports, limited staff.
0:30–1:15	Hindi/Odia voice intake → transcript → structured information.
1:15–2:00	Synthetic report → quality check → OCR → extracted values.
2:00–2:45	Triage card → timeline → missing information → follow-up questions → review priority.
2:45–3:30	Reviewer edits/approves/escalates.
3:30–4:00	One-click referral note.
4:00–4:30	Consent, anonymization and audit trail.
4:30–5:00	Show impact and close: “We don't replace the doctor. We make sure the doctor receives better information, faster.”

47. Judge-Wow Differentiators

- Multilingual voice-to-structured triage.
- Medical report OCR with source transparency.
- Explicit missing-information detection — the system knows what it does not know.
- Rules-first safety layer rather than allowing the LLM to make clinical decisions.
- Human-editable triage card.
- One-click referral preparation.
- Visible audit trail.
- Offline/low-connectivity workflow.
- India-specific facility and language context.

48. What NOT to Build

- AI disease diagnosis
- AI doctor chatbot
- Medicine recommendation
- Prescription generation
- Autonomous treatment decisions
- Claims of clinical accuracy without validation
- Real patient data processing
- Unnecessary hardware
- Complex features that do not improve the triage workflow

49. Folder Structure

```

app/
  page.tsx
  consent/
  intake/
  voice/
  reports/
  triage/
  reviewer/
  referral/
  audit/
  settings/

```

```
components/  
  ui/  
  layout/  
  intake/  
  voice/  
  reports/  
  triage/  
  reviewer/  
  queue/  
  referral/  
  privacy/  
  audit/  
  
lib/  
  ai/  
  ocr/  
  speech/  
  translation/  
  safety/  
  db/  
  auth/  
  audit/  
  
db/  
  schema/  
  migrations/  
  
types/  
utils/  
public/
```

50. Master Web-Coding Platform Prompt

You are a senior full-stack engineer, AI engineer, healthcare product architect, UX designer, cybersecurity engineer, and hackathon reviewer.

Build the complete web application “SwasthyaFlow” from this PRD.

IMPORTANT PRODUCT RULE:

This is NOT an AI doctor. It is a human-in-the-loop healthcare information organization and triage-support workflow.

The application must:

- Collect patient information through text and voice.
- Support English, Hindi and Odia demonstration workflows.
- Convert voice to text.
- Preserve original language input.
- Translate/normalize when necessary.
- Upload synthetic medical reports.
- Run OCR/document extraction.
- Preserve raw OCR.
- Extract structured information.
- Build symptom timelines.
- Identify missing and ambiguous information.
- Generate reviewer-facing follow-up questions.
- Run a configurable RULES-BASED safety/review-priority engine.

- Produce GREEN/YELLOW/RED review priority only as an advisory workflow signal.
- Never diagnose.
- Never prescribe.
- Never recommend treatment.
- Never replace a qualified professional.
- Provide a reviewer dashboard.
- Allow every AI-generated field to be edited.
- Support approve, request-information and escalate actions.
- Generate structured referral notes.
- Maintain consent and anonymization.
- Maintain role-based access.
- Maintain an audit trail.
- Use synthetic data only.
- Provide low-connectivity/offline simulation.

ARCHITECTURE:

Use a modular frontend/backend architecture.

Keep AI, OCR, speech, translation and safety logic in separate services/modules.

The LLM should produce structured information, not final clinical decisions.

The rules engine determines configured review-priority signals.

The human reviewer remains the final authority.

UX:

Build a polished, premium healthcare/public-sector interface.

Use responsive layouts, accessible typography, clear status badges, clean cards, useful loading states, empty states and error states.

The Triage Card is the most important screen.

The reviewer dashboard should feel like a real operational queue, not a generic analytics dashboard.

DEMO:

Make the entire application work with a seeded synthetic dataset so the complete workflow can be demonstrated even if external AI APIs are unavailable.

Provide realistic mock fallbacks for OCR, transcription and AI processing while keeping the integration points ready for real APIs.

Do not fake successful persistence when the database fails.

SAFETY:

Every health-related AI output must include a clear disclaimer that it is advisory, non-diagnostic and reviewer-facing.

Never invent patient information.

Unknown information must remain unknown.

Ambiguous information must remain ambiguous.

Show source information wherever practical.

BUILD QUALITY:

Use reusable components.

Use strong TypeScript typing.

Use validation.

Handle API failures gracefully.

Do not expose API keys in client code.

Do not hard-code sensitive credentials.

Provide clear setup instructions and environment variables.

Provide database migrations/schema.

Provide seed data.

Provide a README.

Ensure the application can be run locally and deployed through a GitHub-connected web platform.

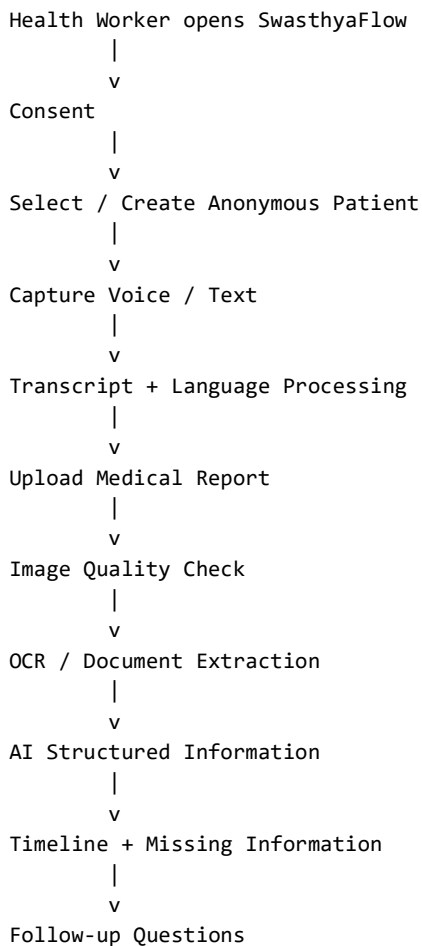
FIRST BUILD PRIORITIES:

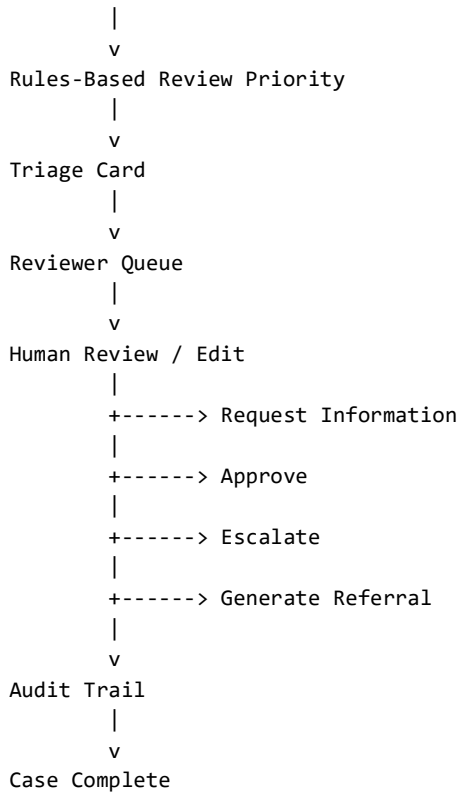
1. App shell and navigation
2. Consent
3. Patient intake
4. Voice/text interface
5. Report upload
6. OCR/AI mock pipeline
7. Triage card
8. Reviewer queue
9. Review/edit/escalate
10. Referral
11. Audit
12. Privacy/settings
13. Offline simulation

Do not build a generic chatbot.

Build the complete healthcare workflow described in this PRD.

51. Final Product Flow





52. Product Positioning

Do not pitch SwasthyaFlow as an AI doctor.

Recommended positioning:

“SwasthyaFlow is an AI-assisted frontline triage and referral-support layer for India's distributed healthcare facilities.”

Final pitch:

“We are not building an AI doctor. We are building the information layer between the patient and the doctor.”

53. Prototype Disclaimer

This application is an educational hackathon prototype for triage-support and information organization. It is not a medical device or diagnostic system. It does not provide medical diagnosis, treatment, prescriptions, or clinical decisions. All health-related information must be reviewed by qualified healthcare professionals.

54. Source Basis

This PRD is based on the supplied hackathon problem statement and follows its requested emphasis on safety-first triage, information extraction and summarization, multimodal capability, India-wide relevance, human review and escalation, privacy, and responsible AI.

The supplied problem statement also requires synthetic/public sample data, a clear educational prototype disclaimer, consent, minimal data retention, anonymization, auditability, and reviewer-facing advisory outputs.

55. AI/ML Component Table

The AI/ML layer must be modular, auditable, and responsibility-separated. Each component has a defined role and must expose uncertainty or failure rather than silently producing fabricated information.

Component	Technology	Responsibility	Required Behavior
Speech-to-Text	OpenAI Whisper	Transcribe patient/health-worker voice.	Preserve transcript; expose uncertainty; never determine risk.
OCR	Tesseract OCR	Extract raw text from uploaded reports after preprocessing.	Preserve raw OCR; uncertain tokens use the “Possibly” prefix.
LLM / Information Extraction	Google Gemini Flash	Structure symptoms, timelines, report information, missing information and follow-up questions.	JSON output only; no diagnosis, treatment, prescription, or final risk decision.
Indian Language Translation	IndicTrans2	Translate/normalize Hindi, Odia and other supported Indian-language content.	Preserve original language; do not discard source text.

56. AI/ML Boundaries

- Whisper performs transcription only.
- Tesseract performs OCR only.
- IndicTrans2 performs language translation/normalization only.
- Gemini Flash organizes and structures information only.
- The deterministic rules engine owns the final configured review-priority result.
- No AI component may diagnose, prescribe, recommend treatment, or replace a qualified professional.

57. HARD SAFETY RULES — NON-NEGOTIABLE

These are backend-enforced product requirements. They must not depend only on frontend behavior.

- LLM CANNOT LOWER RISK: An LLM response can never downgrade a priority generated by the safety rules engine.
- RULES ENGINE WINS: If the LLM and deterministic rules disagree, the rules engine wins.
- TIES GO TOWARD CAUTION: If signals conflict, select the higher priority: RED > YELLOW > GREEN.
- NO SILENT DOWNGRADE: Re-running AI, refreshing a page, or receiving a later model response cannot silently reduce priority.
- HUMAN OVERRIDE ONLY: A qualified authorized reviewer may override the displayed priority through an explicit override action.
- MANDATORY OVERRIDE REASON: Any risk downgrade/override requires a written reason.
- NO FABRICATION: Unknown information remains unknown; ambiguous information remains ambiguous.
- NO DIAGNOSIS: The system must not output definitive disease diagnoses, treatment plans, prescriptions, or medication recommendations.

58. Risk Resolution Logic

Priority precedence:
RED > YELLOW > GREEN

Examples:

```
LLM = GREEN + Rules = RED      -> FINAL = RED
LLM = YELLOW + Rules = RED     -> FINAL = RED
Rule A = GREEN + Rule B = YELLOW -> FINAL = YELLOW
Rule A = YELLOW + Rule B = RED  -> FINAL = RED
```

If the safety engine cannot produce a valid result:
FINAL = PRIORITY_UNAVAILABLE

Never default an unavailable safety result to GREEN.

The final priority must be calculated server-side.

59. RED-TAG PINNING & MANDATORY OVERRIDE

- Every RED case is pinned above YELLOW and GREEN cases in the reviewer queue.
- RED must remain visually prominent on the triage card and queue.
- AI retries and subsequent model outputs cannot automatically downgrade RED.
- Only an authorized reviewer can override a RED result.
- Downgrading RED requires a mandatory free-text override reason.
- Store previous priority, new priority, reviewer ID, timestamp and reason.
- Display a visible RED OVERRIDE audit event after a successful downgrade.
- Block submission if a RED override reason is empty.

Example audit event:

```
{
  "event": "RISK_OVERRIDE",
  "previous_priority": "RED",
  "new_priority": "YELLOW",
  "reviewer_id": "U-104",
  "reason": "Reviewer verified the source information and corrected the configured signal.",
  "timestamp": "<timestamp>"
}
```

60. PIPELINE-FAILURE RULE

An empty triage note is a hard pipeline failure.

- If triage generation returns null, empty, malformed, or unusable content, set encounter state to TRIAGE_FAILED.
- Never show an empty triage card as a successful result.
- Never save an empty triage note as successful AI processing.
- Never assign GREEN as a fallback.
- Preserve source inputs and intermediate outputs where available.
- Provide retry and manual-review paths.
- Clearly display the failure state to the reviewer.

Successful triage generation requires valid structured output and a non-empty usable triage result. If the safety engine cannot resolve priority, use PRIORITY_UNAVAILABLE rather than inventing a safe result.

61. UNCERTAINTY HANDLING

Uncertainty must be preserved. Do not convert low-confidence OCR, speech recognition, translation, or extraction into false certainty.

Source	Condition	Required Output
Tesseract OCR	Uncertain word/value/medicine name	Prefix uncertain text with “Possibly”.
Whisper ASR	Unclear speech/transcription	Prefix exposed uncertain token/phrase with “Possibly”.
IndicTrans2	Ambiguous translation	Preserve original text and mark translated wording as uncertain/ambiguous.
Gemini extraction	Information is incomplete or inferred	Do not convert inference into fact; use unknown/ambiguous fields.

Examples:

Possibly Levolin

Possibly 120 mg

Possibly dizziness

The original image/audio/transcript must remain available to the reviewer when uncertainty is material.

62. PERFORMANCE TARGETS

Workflow	Target	Required Behavior if Target Is Missed
Patient intake	< 20 seconds	Show progress; allow manual continuation or retry.
OCR	< 15 seconds for normal synthetic reports	Show progress; retry or request a clearer image.
Triage generation	Target < 15 seconds after inputs are ready	Show stage-level progress; preserve intermediate data.
Reviewer dashboard	Target < 2 seconds for normal seeded/demo data	Use pagination/filtering and avoid unnecessary client work.

These are prototype performance targets, not clinical-service guarantees.

63. FAILURE-STATE MATRIX

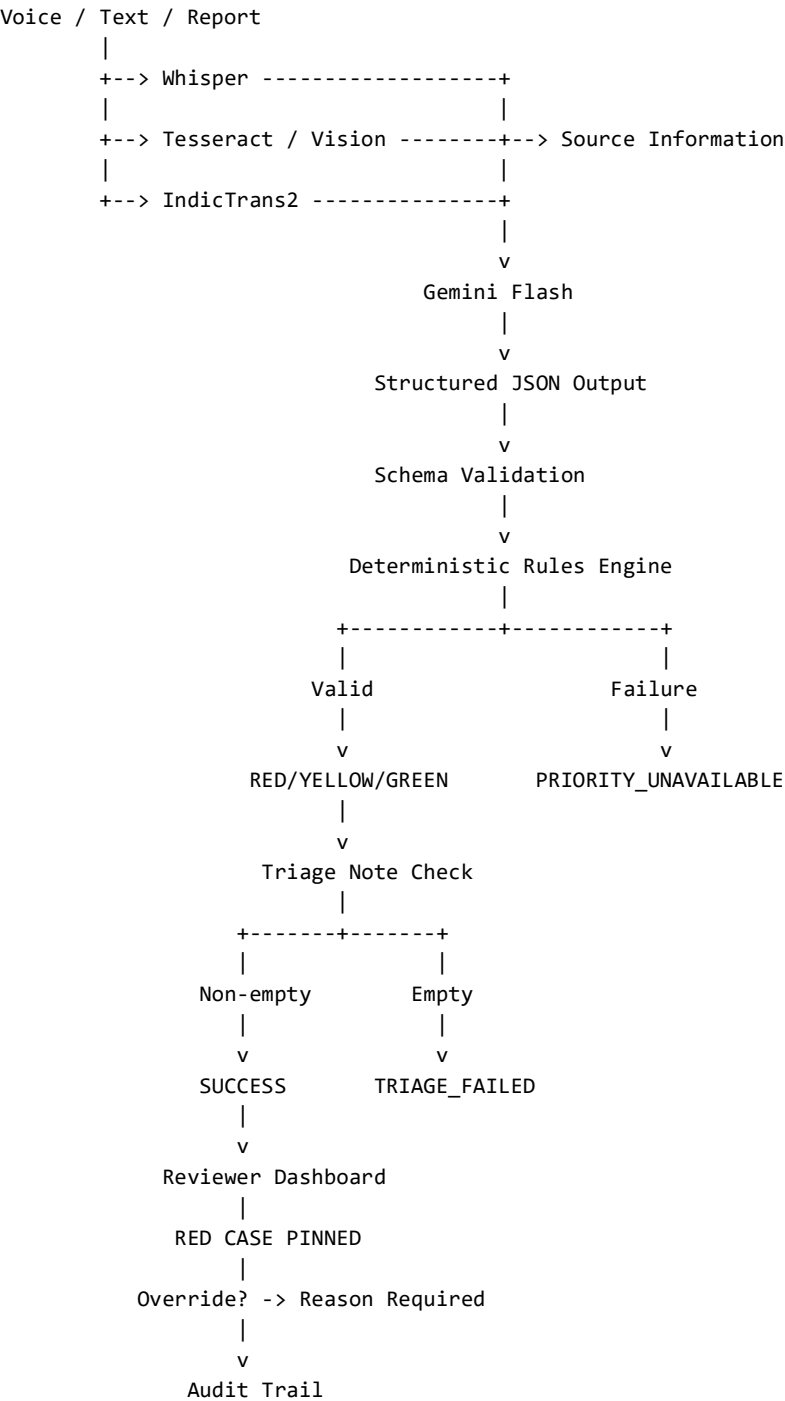
Failure	State	Behavior
Speech service unavailable	ASR_FAILED	Retry/manual text entry; never fabricate transcript.
OCR unavailable	OCR_FAILED	Preserve image; retry/manual entry.
LLM unavailable	AI_FAILED	Preserve source data; manual reviewer workflow.
Empty triage note	TRIAGE_FAILED	Hard failure; never GREEN; retry/manual path.
Invalid AI JSON	AI_PARSE_FAILED	Retry/repair; do not silently continue.
Rules engine unavailable	SAFETY_ENGINE_FAILED	Never default to GREEN; priority unavailable.
Database save failure	SAVE_FAILED	Never claim save succeeded; allow retry.
Network lost	OFFLINE/PENDING_SYNC	Queue safely and show sync status.

64. Security-Critical Implementation Requirements

- Enforce safety-priority resolution on the backend.
- Do not allow client requests to directly set a lower priority.
- Enforce mandatory RED override reason server-side.

- Audit every priority change and override.
- Validate all Gemini output against a strict JSON schema.
- Treat model output as untrusted input.
- Never expose provider API keys in browser code.
- Use synthetic/public data only.
- Do not send unnecessary personal information to AI providers.

65. Updated AI + Safety Pipeline



66. Updated Master Coding-Agent Requirements

Treat the following as hard acceptance criteria when generating the application:

1. Use Whisper for speech-to-text, Tesseract for OCR, Gemini Flash for information extraction/summarization, and IndicTrans2 for Indian-language translation/normalization. If unavailable during development, create clean mock adapters with the same interfaces.
2. Keep all four AI/ML components modular and replaceable.
3. The LLM MUST NEVER lower the risk priority.
4. The deterministic rules engine MUST win whenever it conflicts with model output.
5. Conflicting priorities MUST resolve toward caution: RED > YELLOW > GREEN.
6. An empty, null, malformed or unusable triage note MUST be TRIAGE_FAILED.
7. TRIAGE_FAILED MUST NEVER fall back to GREEN.
8. Uncertain OCR/ASR content MUST be marked with “Possibly” and remain linked to the original source.
9. Patient intake target is <20 seconds under normal prototype conditions.
10. OCR target is <15 seconds for a normal synthetic report.
11. RED cases MUST be pinned at the top of the reviewer queue.
12. RED cannot be automatically downgraded.
13. A RED override MUST require an authorized human reviewer and a mandatory written reason.
14. Every override MUST be recorded in the audit trail.
15. All safety enforcement MUST exist server-side, not only in frontend code.
16. The AI must never diagnose, prescribe, recommend treatment, or replace qualified healthcare staff.
17. Use synthetic/public sample data only.
18. Provider failures must produce explicit failure states and safe retry/manual paths.
19. Build a complete operational workflow, not a generic chatbot.

67. Final Acceptance Checklist

- Whisper transcription works or has a mock adapter.
- Tesseract OCR works or has a mock adapter.
- Gemini Flash structured extraction works or has a mock adapter.
- IndicTrans2 translation/normalization works or has a mock adapter.
- Original voice/text/report sources remain available.
- Uncertain OCR/ASR content uses the “Possibly” prefix.
- LLM cannot lower risk.
- Rules engine wins conflicts.
- Ties resolve toward caution.
- Empty triage note becomes TRIAGE_FAILED.
- TRIAGE_FAILED never becomes GREEN.
- RED cases are pinned.
- RED override requires a reason.
- Override is audited.
- Reviewer can edit AI information.
- Referral can be generated.
- Consent and anonymization are visible.
- Performance targets are measurable in demo/testing.
- All demo data is synthetic.

Engineering note: These additions are intentionally stricter than ordinary AI-app requirements because the hackathon awards 20% for safety-first triage and 15% for human-review/escalation. The architecture should make unsafe behavior structurally difficult rather than relying on prompts alone.